Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#06)

Use of Venus Simulator for RISC-V Procedure Execution

Student Name: Manahil Shahzad Roll No: 24I-6563 Section: CE-A

Date performed: 28TH September, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1: Examine why the below procedure is not displaying correct value in t1 register. Fix it.

Description:

In this task, we examined and corrected a procedure that was not producing the correct value in register t1. We used jal to call the procedure and jalr to return to the main program. The procedure calculates 3+1000=1003, which is returned in a0. Adding this to 2500 gives the final result 3503 in t1.

Code:

```
.text

.globl main



main:

   li a0, 1

   li s0, 2500

   jal ra, procedure

   add t1, s0, a0



   li a0, 1

   addi a1, t1, 0

   ecall



   li a0, 10

   ecall



procedure:
```

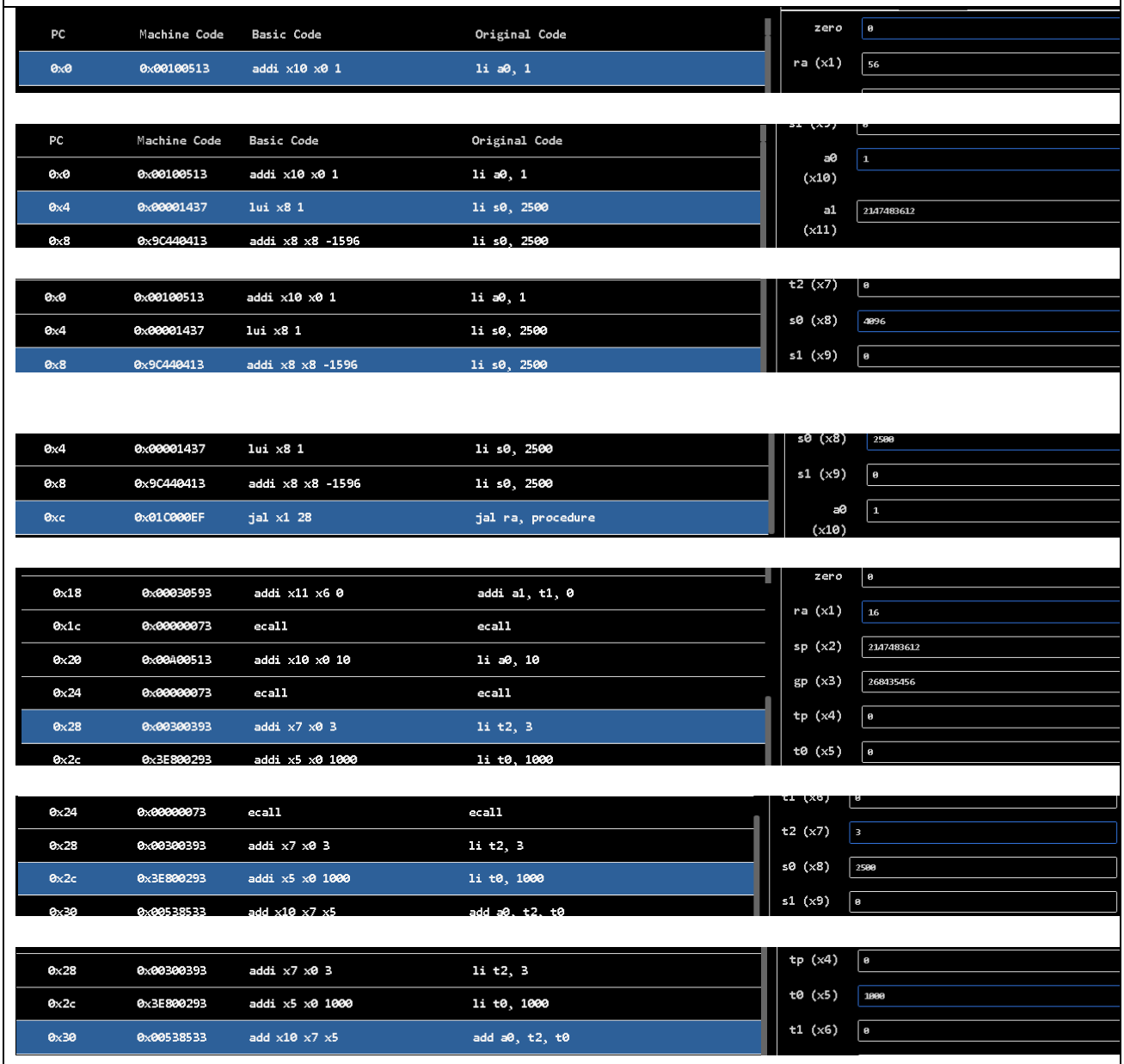
```
li t2, 3

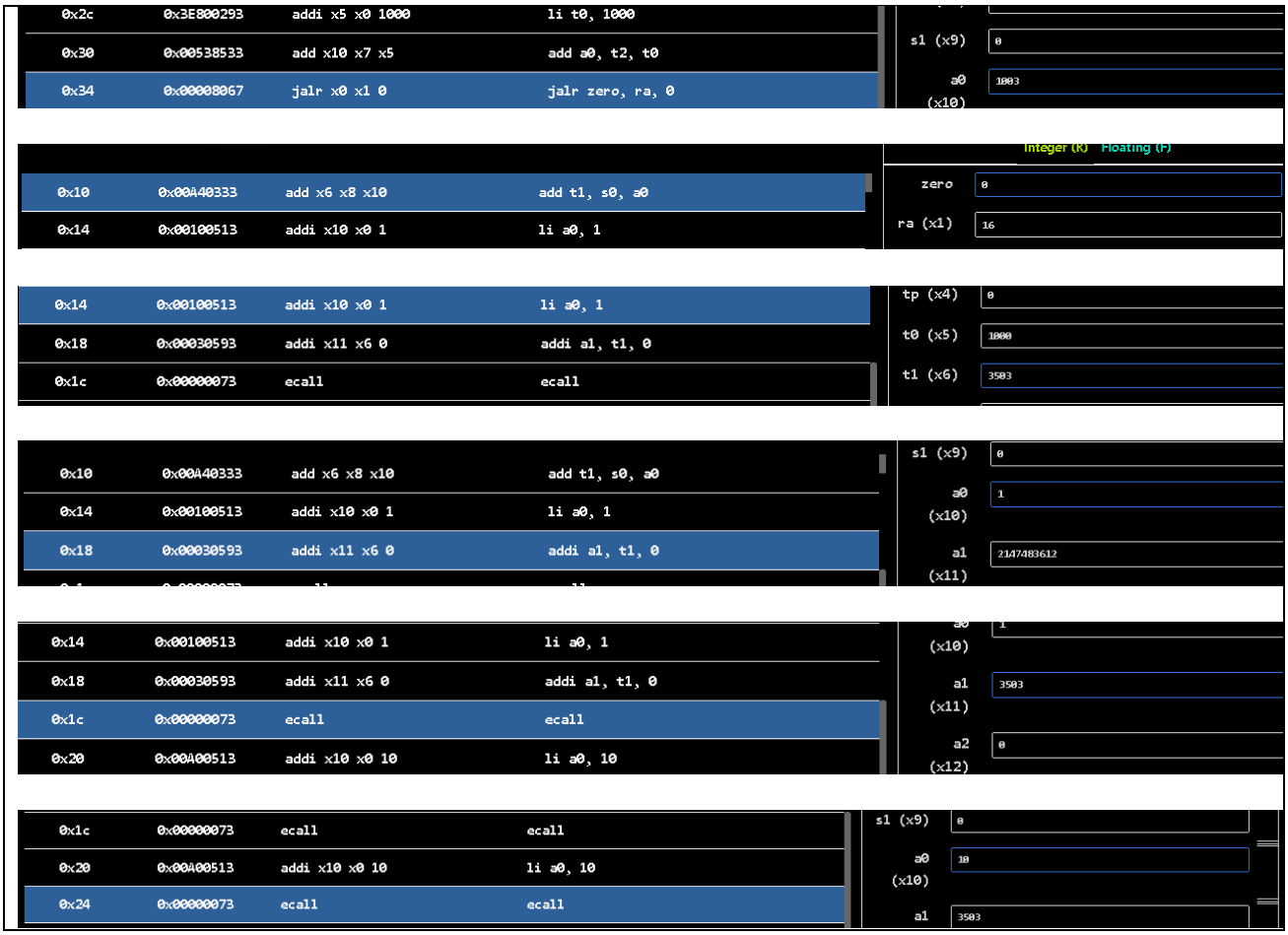
li t0, 1000

add a0, t2, t0

jalr zero, ra, 0
```

Output:




0x2c 0x3E800293 addi x5 x0 1000 li t0, 1000
0x30 0x00538533 add x10 x7 x5 add a0, t2, t0
0x34 0x00008067 jalr x0 x1 0 jalr zero, ra, 0
s1 (x9) 0
a0 (x10) 1003
Integer (R) Floating (F)
0x10 0x00A40333 add x6 x8 x10 add t1, s0, a0
0x14 0x00100513 addi x10 x0 1 li a0, 1
zero 0
ra (x1) 16
0x14 0x00100513 addi x10 x0 1 li a0, 1
0x18 0x00030593 addi x11 x6 0 addi a1, t1, 0
0x1c 0x00000073 ecall ecall
tp (x4) 0
t0 (x5) 1000
t1 (x6) 3503
0x10 0x00A40333 add x6 x8 x10 add t1, s0, a0
0x14 0x00100513 addi x10 x0 1 li a0, 1
0x18 0x00030593 addi x11 x6 0 addi a1, t1, 0
s1 (x9) 0
a0 (x10) 1
a1 (x11) 2147483612
0x14 0x00100513 addi x10 x0 1 li a0, 1
0x18 0x00030593 addi x11 x6 0 addi a1, t1, 0
0x1c 0x00000073 ecall ecall
0x20 0x00A00513 addi x10 x0 10 li a0, 10
a0 (x10) 1
a1 (x11) 3503
a2 (x12) 0
0x1c 0x00000073 ecall ecall
0x20 0x00A00513 addi x10 x0 10 li a0, 10
0x24 0x00000073 ecall ecall
s1 (x9) 0
a0 (x10) 10
a1 3503

Task 2: Run given example on Venus Simulator and explain the code.

Description:

In this task, we initialized registers s0, t1, and t0 with the values 5, 10, and 15 using the addi instruction. We allocated 12 bytes of stack space and used sw to store the values in memory. We then used lw to load them into registers t4, t2, and t3 and restored the stack pointer. The final values are **t4 = 10, t2 = 15, and t3 = 5**.

Code:

```
addi s0, zero, 5

addi t1, zero, 10

addi t0, zero,15

addi sp, sp, -12

sw   s0, 0(sp)

sw   t0, 4(sp)

sw   t1, 8(sp)

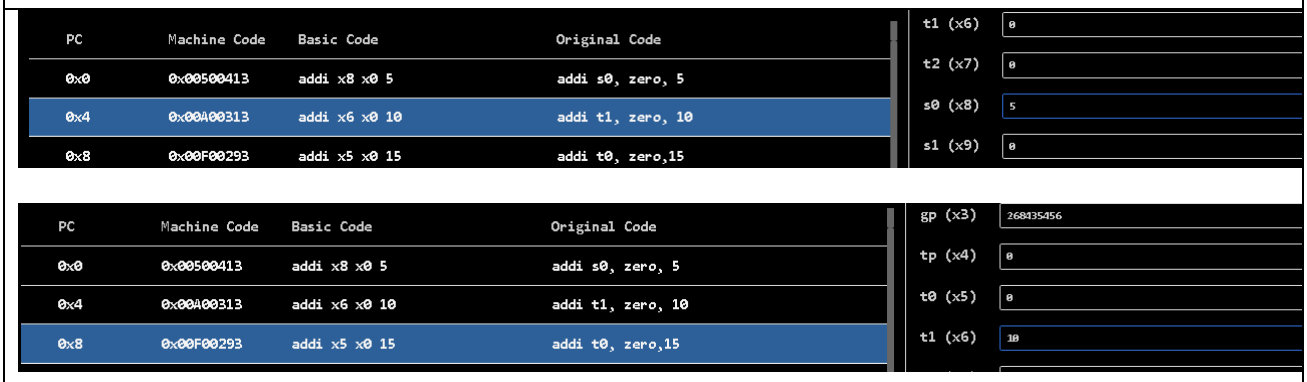
lw   t4, 8(sp)

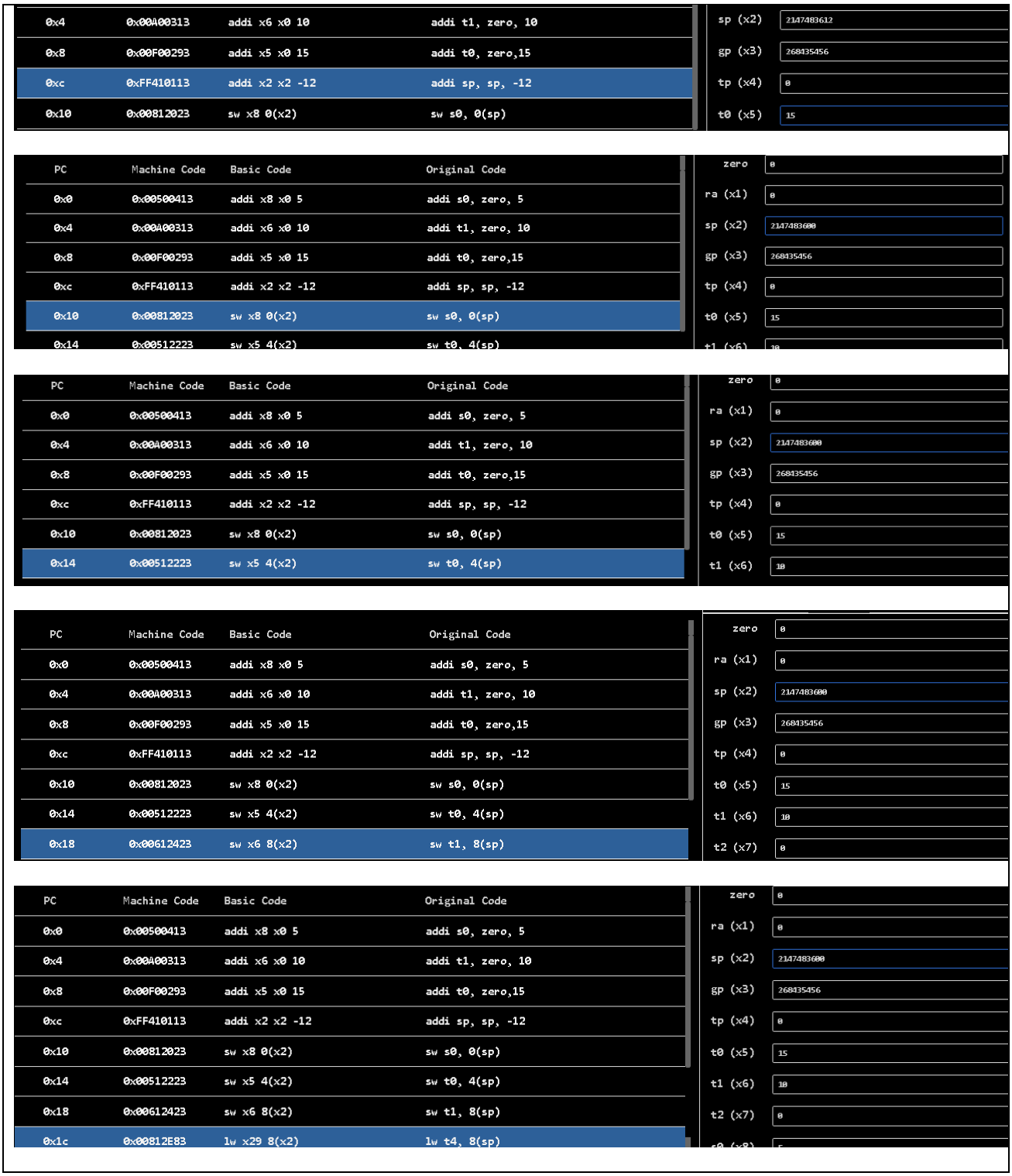
lw   t2, 4(sp)

lw   t3, 0(sp)

addi sp, sp, 12
```

Output:





Task 3: Write RISC-V assembly code for given C code:

Description:

This code executes a mathematical solver and calculates the expression (a+b)(a-b). We have stored two values 5 and 3 in two registers and also used the jalr instruction to jump back to the main function using the saved address in ra.

Code:

```
.text

.globl main

main:

   li a0, 5

   li a1, 3

   jal ra, EqSolver

   addi s0, a0, 0

   li a0, 1

   addi a1, s0, 0

   ecall

   li a0, 10

   ecall

EqSolver:

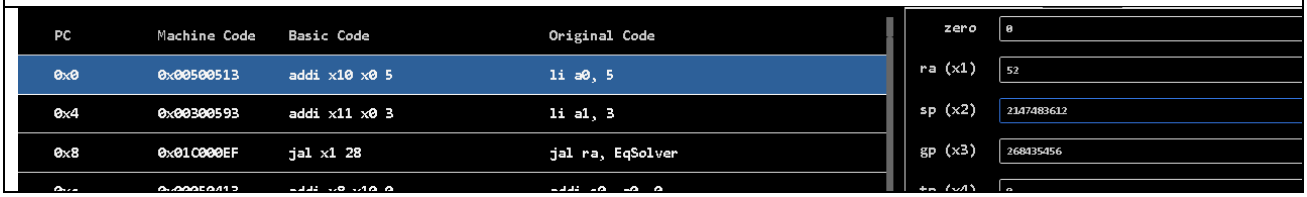
   add t0, a0, a1

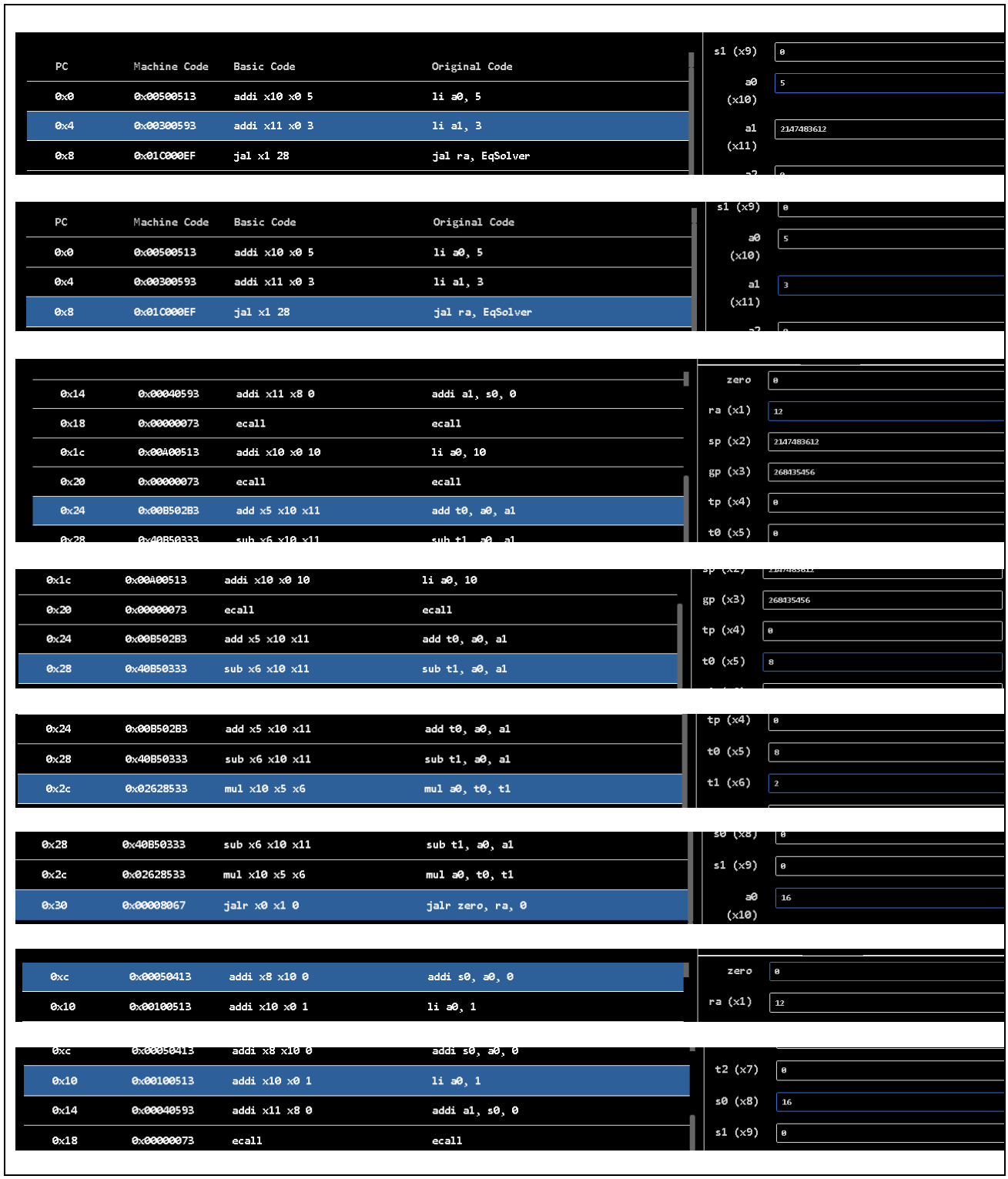
   sub t1, a0, a1

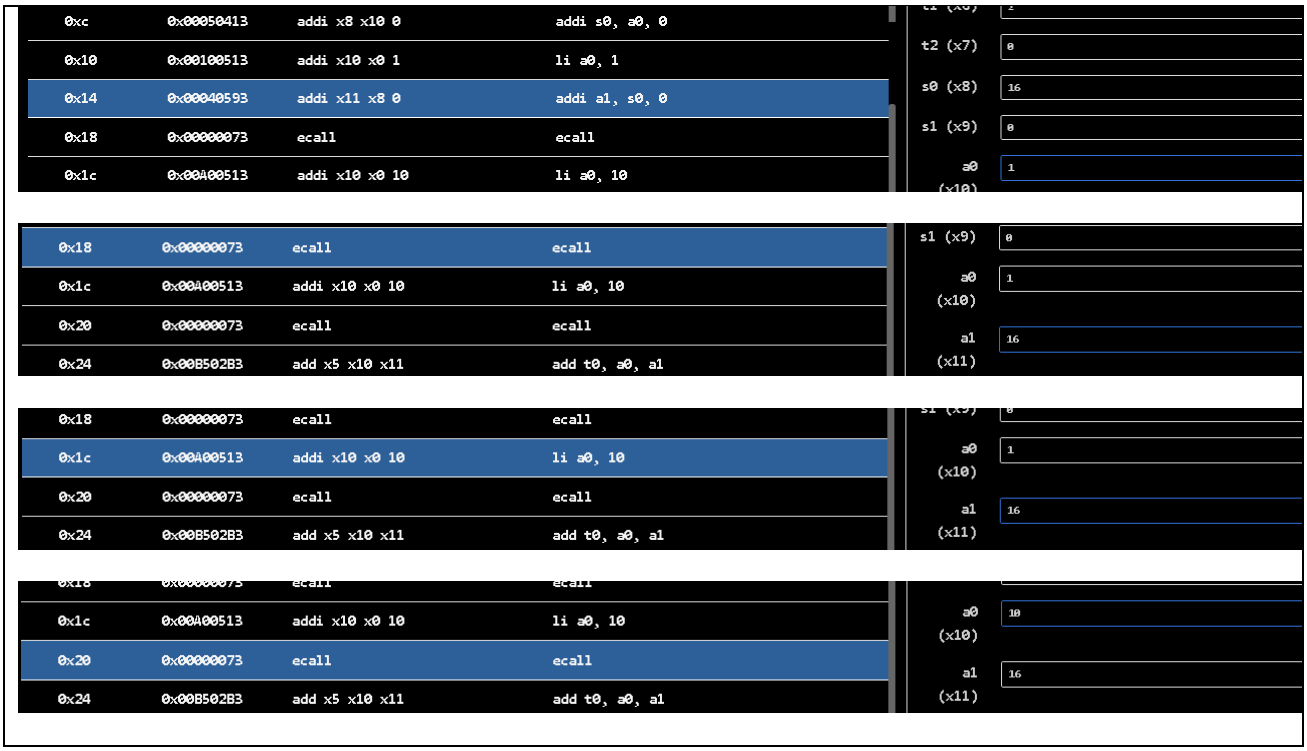
   mul a0, t0, t1

   jalr zero, ra, 0
```

Output:




PC Machine Code Basic Code Original Code
0x0 0x00500513 addi x10 x0 5 li a0, 5
0x4 0x00300593 addi x11 x0 3 li a1, 3
0x8 0x01C000EF jal x1 28 jal ra, EqSolver
s1 (x9) 0
a0 (x10) 5
a1 (x11) 2147483612
PC Machine Code Basic Code Original Code
0x0 0x00500513 addi x10 x0 5 li a0, 5
0x4 0x00300593 addi x11 x0 3 li a1, 3
0x8 0x01C000EF jal x1 28 jal ra, EqSolver
s1 (x9) 0
a0 (x10) 5
a1 (x11) 3
0x14 0x00040593 addi x11 x8 0 addi a1, s0, 0
0x18 0x00000073 ecall ecall
0x1c 0x00A00513 addi x10 x0 10 li a0, 10
0x20 0x00000073 ecall ecall
0x24 0x00B502B3 add x5 x10 x11 add t0, a0, a1
zero 0
ra (x1) 12
sp (x2) 2147483612
gp (x3) 268435456
tp (x4) 0
t0 (x5) 0
0x1c 0x00A00513 addi x10 x0 10 li a0, 10
0x20 0x00000073 ecall ecall
0x24 0x00B502B3 add x5 x10 x11 add t0, a0, a1
0x28 0x40B50333 sub x6 x10 x11 sub t1, a0, a1
gp (x3) 268435456
tp (x4) 0
t0 (x5) 8
0x24 0x00B502B3 add x5 x10 x11 add t0, a0, a1
0x28 0x40B50333 sub x6 x10 x11 sub t1, a0, a1
0x2c 0x02628533 mul x10 x5 x6 mul a0, t0, t1
tp (x4) 0
t0 (x5) 8
t1 (x6) 2
0x28 0x40B50333 sub x6 x10 x11 sub t1, a0, a1
0x2c 0x02628533 mul x10 x5 x6 mul a0, t0, t1
0x30 0x00008067 jalr x0 x1 0 jalr zero, ra, 0
s1 (x9) 0
a0 (x10) 16
0xc 0x00050413 addi x8 x10 0 addi s0, a0, 0
0x10 0x00100513 addi x10 x0 1 li a0, 1
zero 0
ra (x1) 12
0xc 0x00050413 addi x8 x10 0 addi s0, a0, 0
0x10 0x00100513 addi x10 x0 1 li a0, 1
0x14 0x00040593 addi x11 x8 0 addi a1, s0, 0
0x18 0x00000073 ecall ecall
t2 (x7) 0
s0 (x8) 16
s1 (x9) 0



Task 4: Write RISC-V assembly code to find the area and perimeter of square. You need to write two procedures (i.e. AreaFinder and PerimeterFinder) and call both functions in main module.

Description:
In this task, we created two procedures, AreaFinder and PerimeterFinder, to calculate the area and perimeter of a square with side length 5. We passed the side length through register a0 and used multiplication to calculate both values. The results were stored in registers s1 and s2. The final area is 25 square units, and the perimeter is 20 units.
Code:
.text .globl main main: li s0, 5

```
    addi a0, s0, 0

    jal ra, AreaFinder

    addi s1, a0, 0

    addi a0, s0, 0

    jal ra, PerimeterFinder

    addi s2, a0, 0

    li a0, 1

    addi a1, s1, 0

    ecall

    li a0, 1

    addi a1, s2, 0

    ecall

    li a0, 10

    ecall

AreaFinder:

    mul a0, a0, a0

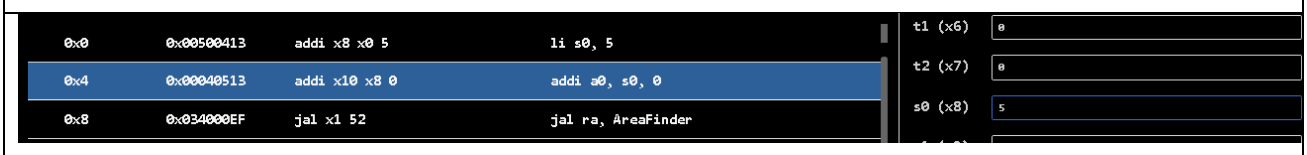
    jalr zero, ra, 0

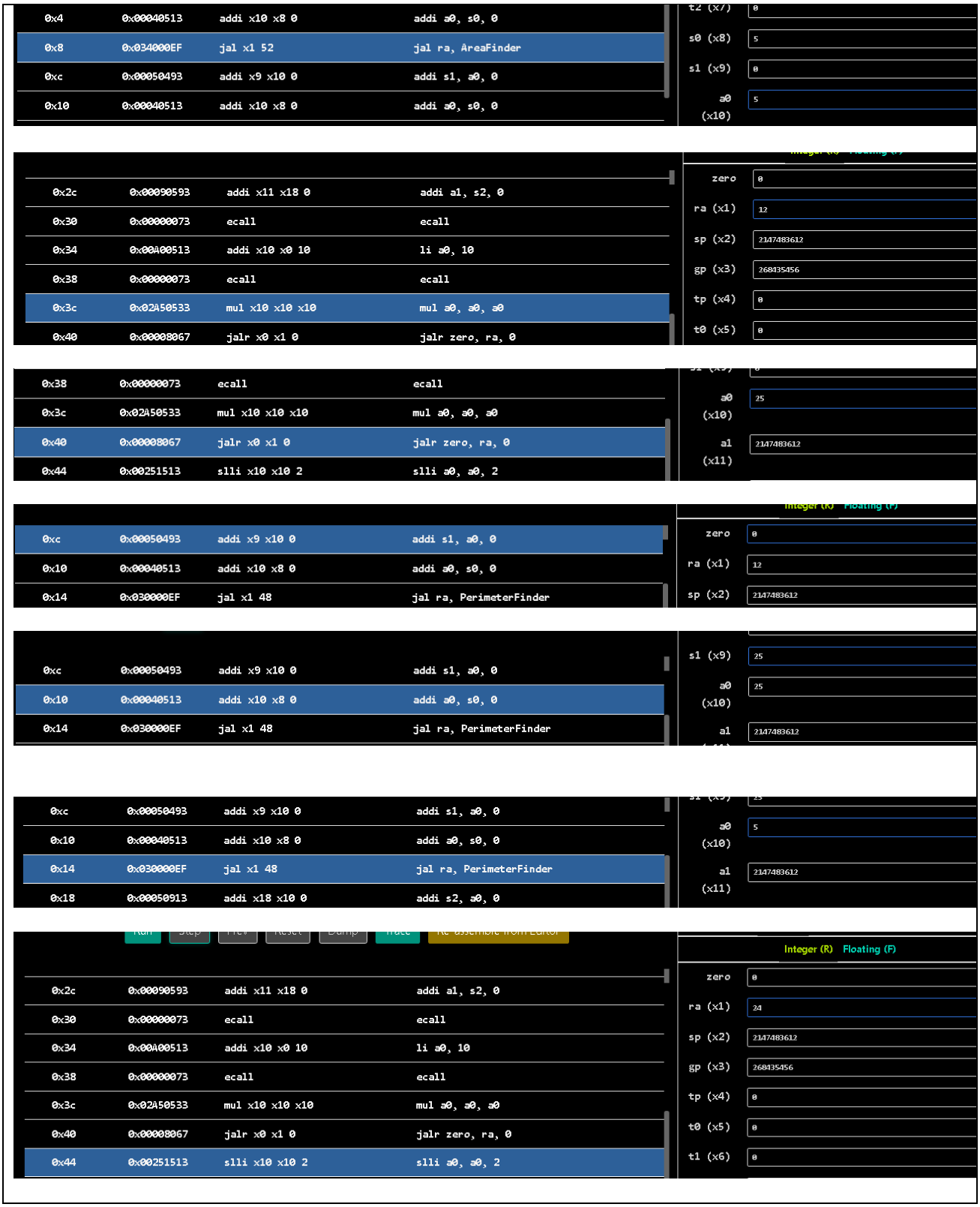
PerimeterFinder:

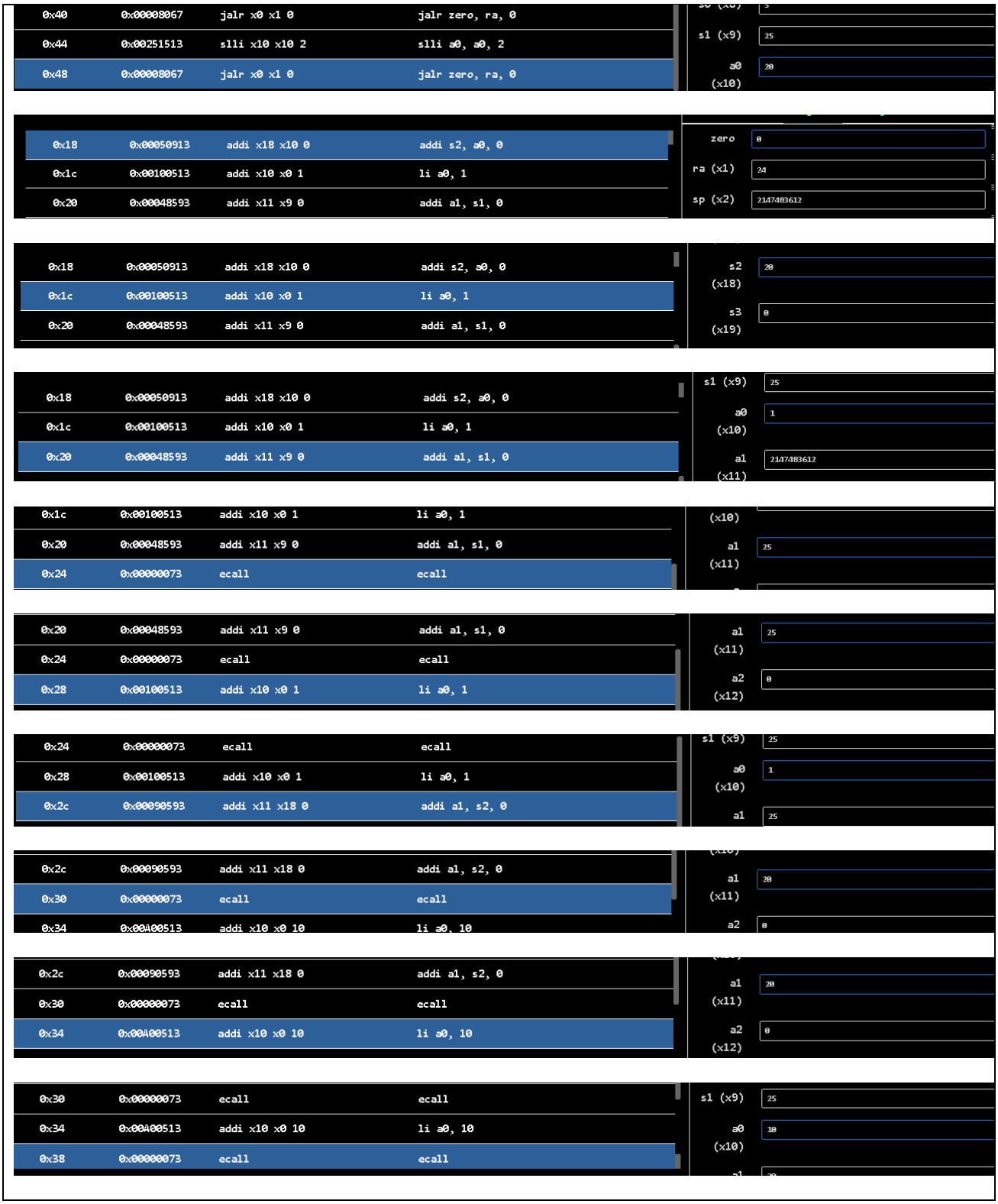
    slli a0, a0, 2

    jalr zero, ra, 0
```

Output:




0x4 0x00040513 addi x10 x8 0 addi a0, s0, 0
0x8 0x034000EF jal x1 52 jal ra, AreaFinder
0xc 0x00050493 addi x9 x10 0 addi s1, a0, 0
0x10 0x00040513 addi x10 x8 0 addi a0, s0, 0
s0 (x8) 5
s1 (x9) 0
a0 (x10) 5
0x2c 0x00090593 addi x11 x18 0 addi a1, s2, 0
0x30 0x00000073 ecall ecall
0x34 0x00A00513 addi x10 x0 10 li a0, 10
0x38 0x00000073 ecall ecall
0x3c 0x02A50533 mul x10 x10 x10 mul a0, a0, a0
0x40 0x00008067 jalr x0 x1 0 jalr zero, ra, 0
zero 0
ra (x1) 12
sp (x2) 2147483612
gp (x3) 268435456
tp (x4) 0
t0 (x5) 0
0x38 0x00000073 ecall ecall
0x3c 0x02A50533 mul x10 x10 x10 mul a0, a0, a0
0x40 0x00008067 jalr x0 x1 0 jalr zero, ra, 0
0x44 0x00251513 slli x10 x10 2 slli a0, a0, 2
a0 (x10) 25
a1 (x11) 2147483612
0xc 0x00050493 addi x9 x10 0 addi s1, a0, 0
0x10 0x00040513 addi x10 x8 0 addi a0, s0, 0
0x14 0x030000EF jal x1 48 jal ra, PerimeterFinder
zero 0
ra (x1) 12
sp (x2) 2147483612
0xc 0x00050493 addi x9 x10 0 addi s1, a0, 0
0x10 0x00040513 addi x10 x8 0 addi a0, s0, 0
0x14 0x030000EF jal x1 48 jal ra, PerimeterFinder
s1 (x9) 25
a0 (x10) 25
a1 2147483612
0xc 0x00050493 addi x9 x10 0 addi s1, a0, 0
0x10 0x00040513 addi x10 x8 0 addi a0, s0, 0
0x14 0x030000EF jal x1 48 jal ra, PerimeterFinder
0x18 0x00050913 addi x18 x10 0 addi s2, a0, 0
a0 (x10) 5
a1 (x11) 2147483612
Integer (R) Floating (F)
0x2c 0x00090593 addi x11 x18 0 addi a1, s2, 0
0x30 0x00000073 ecall ecall
0x34 0x00A00513 addi x10 x0 10 li a0, 10
0x38 0x00000073 ecall ecall
0x3c 0x02A50533 mul x10 x10 x10 mul a0, a0, a0
0x40 0x00008067 jalr x0 x1 0 jalr zero, ra, 0
0x44 0x00251513 slli x10 x10 2 slli a0, a0, 2
zero 0
ra (x1) 24
sp (x2) 2147483612
gp (x3) 268435456
tp (x4) 0
t0 (x5) 0
t1 (x6) 0



Task 5: Write RISC-V assembly code for given C code:

Description:

In this task, we implemented the polynomial $x^3 + 5x^2 + 6x$using a procedure named EqSolver. We passed $x = 2$through register a0 and used multiplication and addition instructions to calculate the polynomial. The procedure returned the result in a0, which was saved in s0 and displayed on the console. The final answer is 40.

Code:

```
.text

.globl main

main:

   li a0, 2

   jal ra, EqSolver

   addi s0, a0, 0

   li a0, 1

   addi a1, s0, 0

   ecall

   li a0, 10

   ecall

EqSolver:

   addi t0, a0, 0

   mul t1, t0, t0

   mul t2, t1, t0

   li t3, 5

   mul t3, t3, t1

   li t4, 6
```

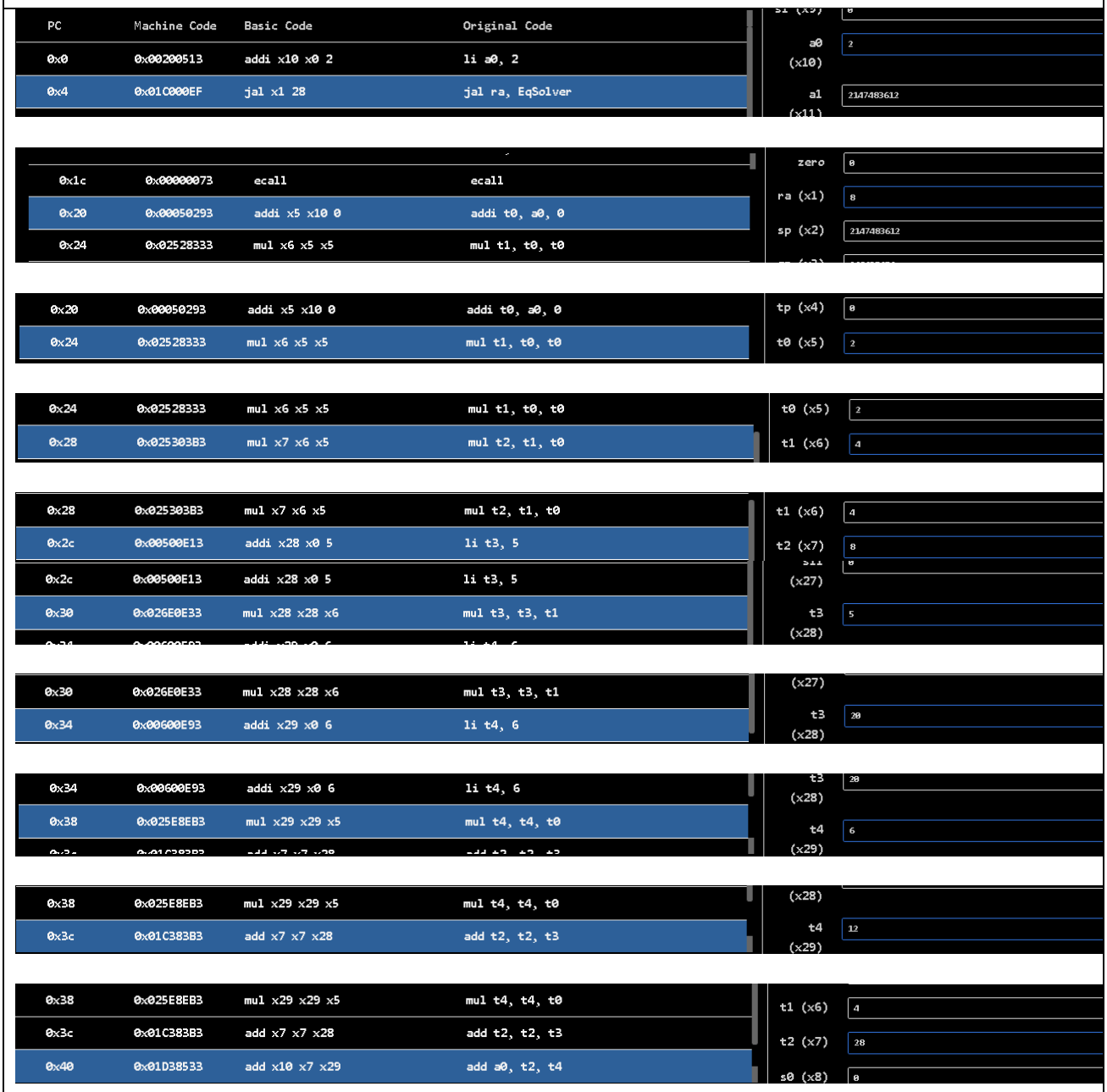
```
mul t4, t4, t0

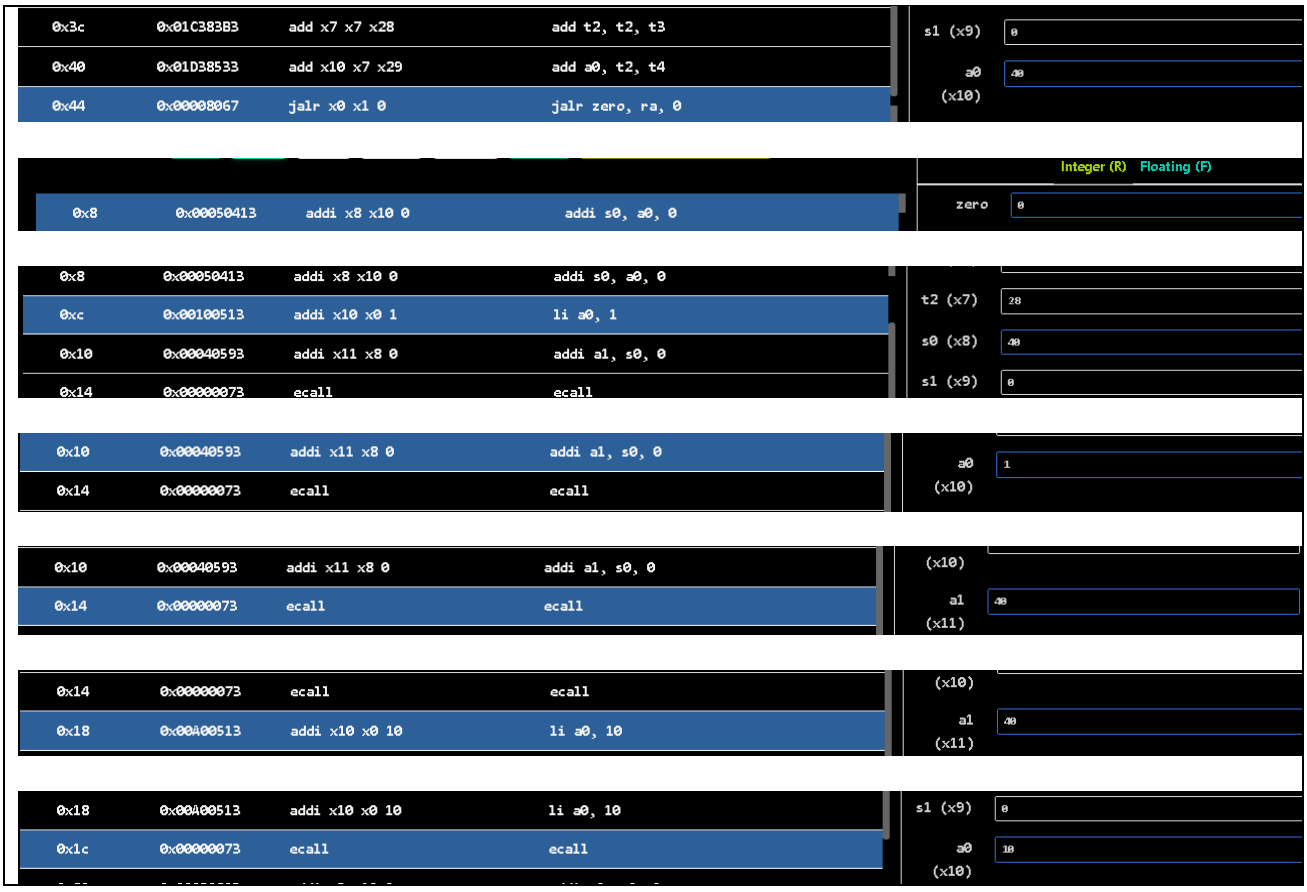
add t2, t2, t3

add a0, t2, t4

jalr zero, ra, 0
```

Output:





Task 6: Implement the following C code in RISC-V assembly using Venus.

Description:
In this task, we used conditional branching instructions beq and blt to select one of three procedures based on the values of s0 and s1. Since s0=3 and s1=2, the program calls procedure F2, which calculates the cube of 3. The result is returned in a0 and stored in s2. The final answer is 27.
Code:
.text .globl main main: li s0, 3

```
    li s1, 2

    li t0, 1

    beq s1, t0, call_F1

    blt s1, s0, call_F2

    j call_F3

call_F1:

    addi a0, s0, 0

    addi a1, s1, 0

    jal ra, F1

    j save_result

call_F2:

    addi a0, s0, 0

    jal ra, F2

    j save_result

call_F3:

    addi a0, s0, 0

    jal ra, F3

save_result:

    addi s2, a0, 0

    li a0, 1

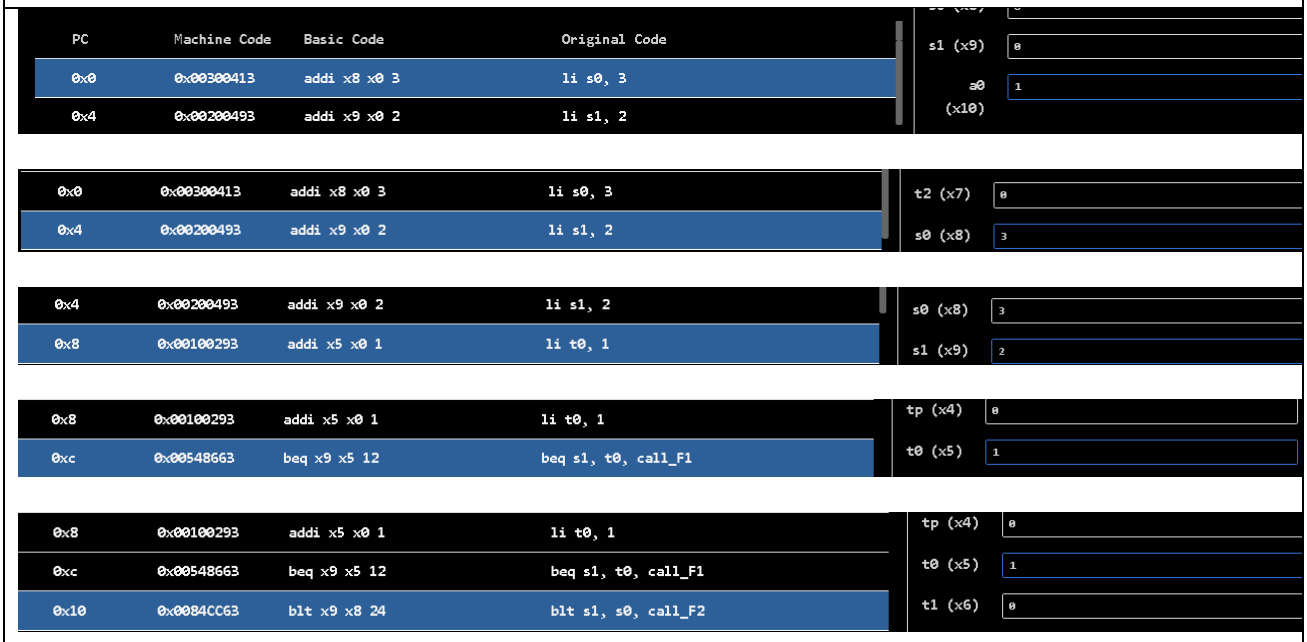
    addi a1, s2, 0

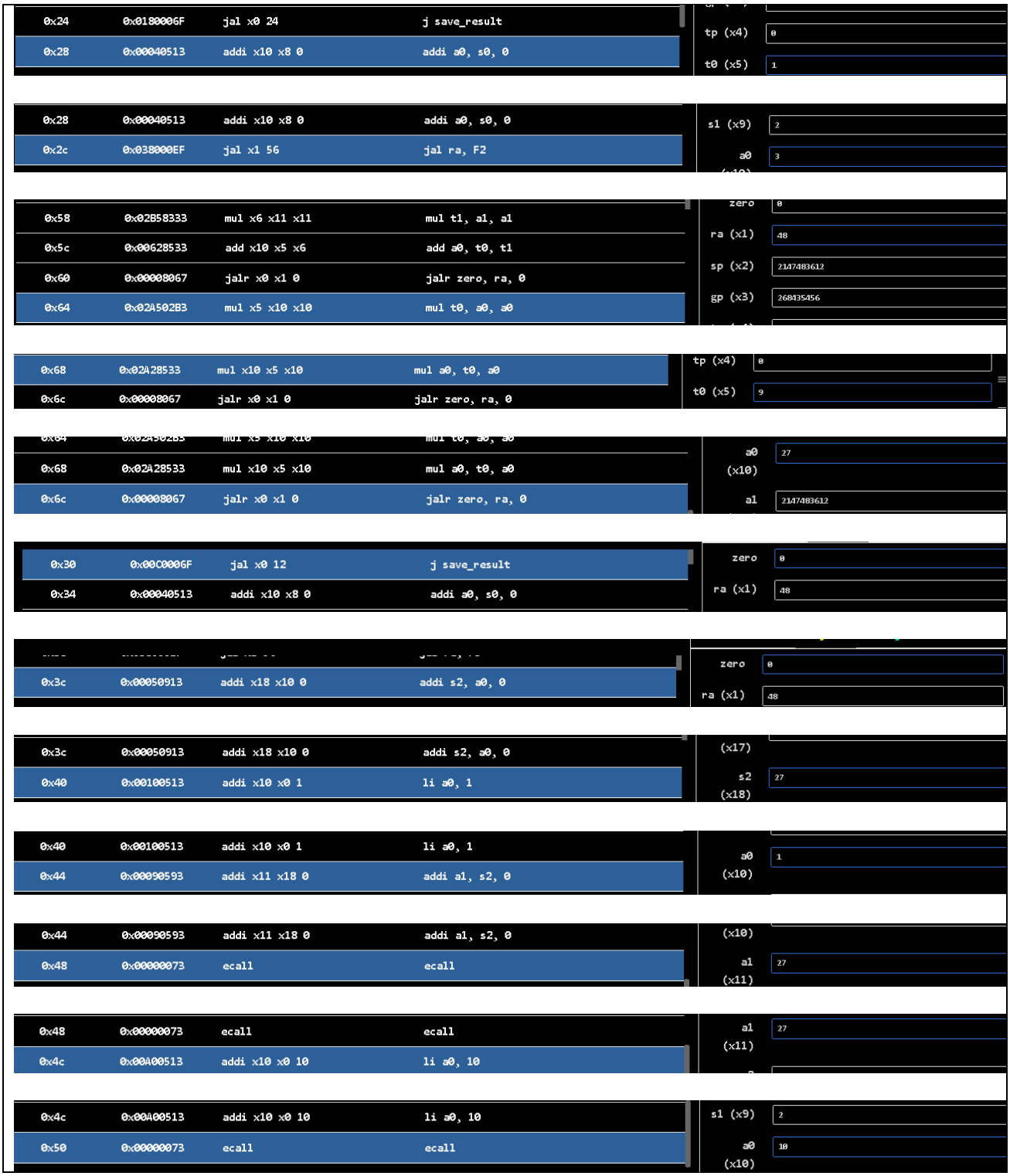
    ecall

    li a0, 10
```

```
    ecall
F1:
    mul t0, a0, a0
    mul t1, a1, a1
    add a0, t0, t1
    jalr zero, ra, 0
F2:
    mul t0, a0, a0
    mul a0, t0, a0
    jalr zero, ra, 0
F3:
    addi a0, a0, 5
    jalr zero, ra, 0
```

Output:





Task 7: Implement the following C code in RISC-V assembly using Venus.

Description:

In this task, we initialized s0 and s1 to 4 and used beq instructions to select the appropriate function. Since s1 is not zero but is equal to s0, the program calls fun2. This function calculates the cube of the input value using multiplication instructions. The final result stored in s2 is 64.

Code:

```
.text

.globl main



main:

   li s0, 4

   li s1, 4

   beq s1, zero, call_fun1

   beq s0, s1, call_fun2

   j call_fun3

call_fun1:

   addi a0, s0, 0

   jal ra, fun1

   j save_result



call_fun2:

   addi a0, s0, 0

   jal ra, fun2

   j save_result
```

```
call_fun3:
    addi a0, s0, 0
    jal ra, fun3

save_result:
    addi s2, a0, 0

    li a0, 1
    addi a1, s2, 0
    ecall

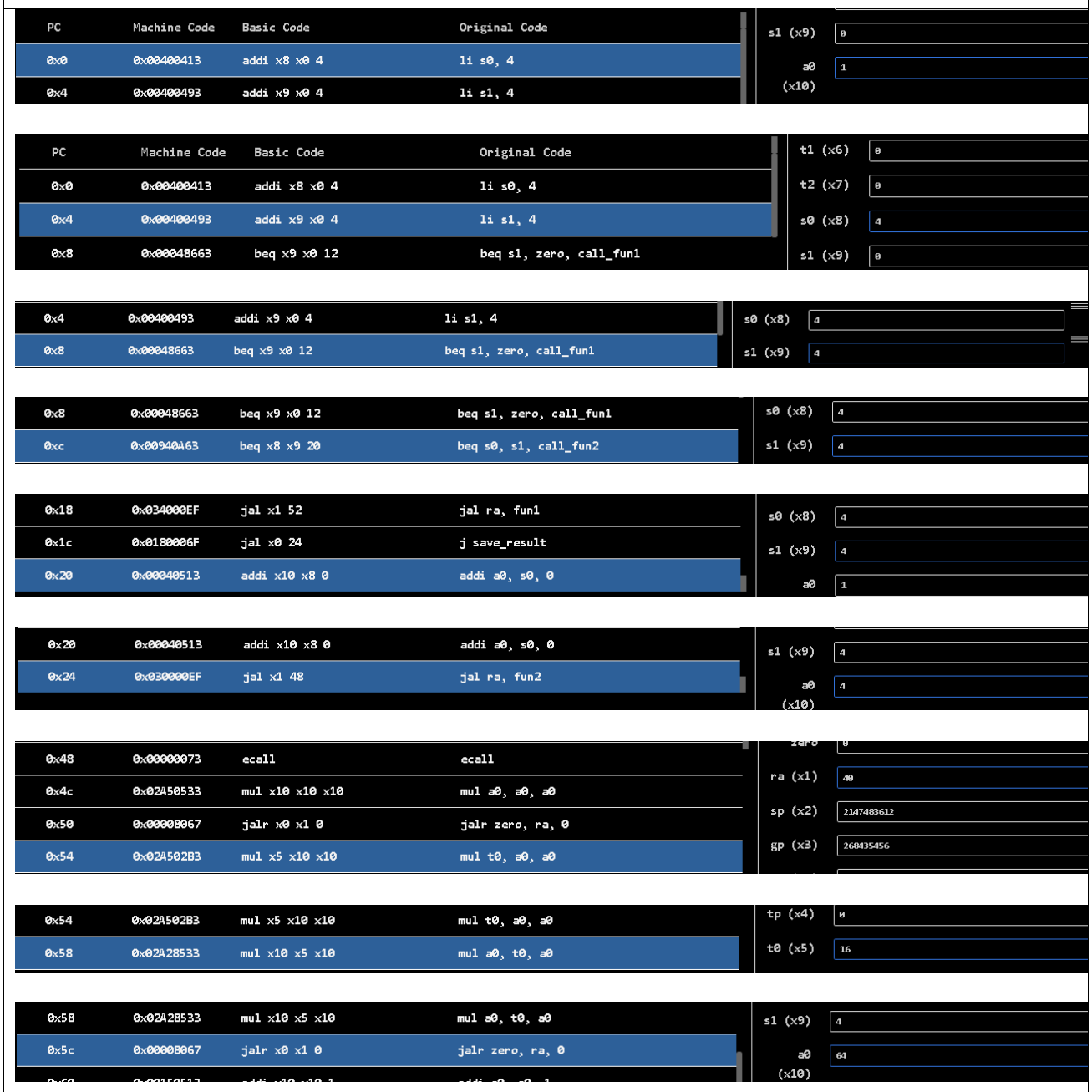
    li a0, 10
    ecall

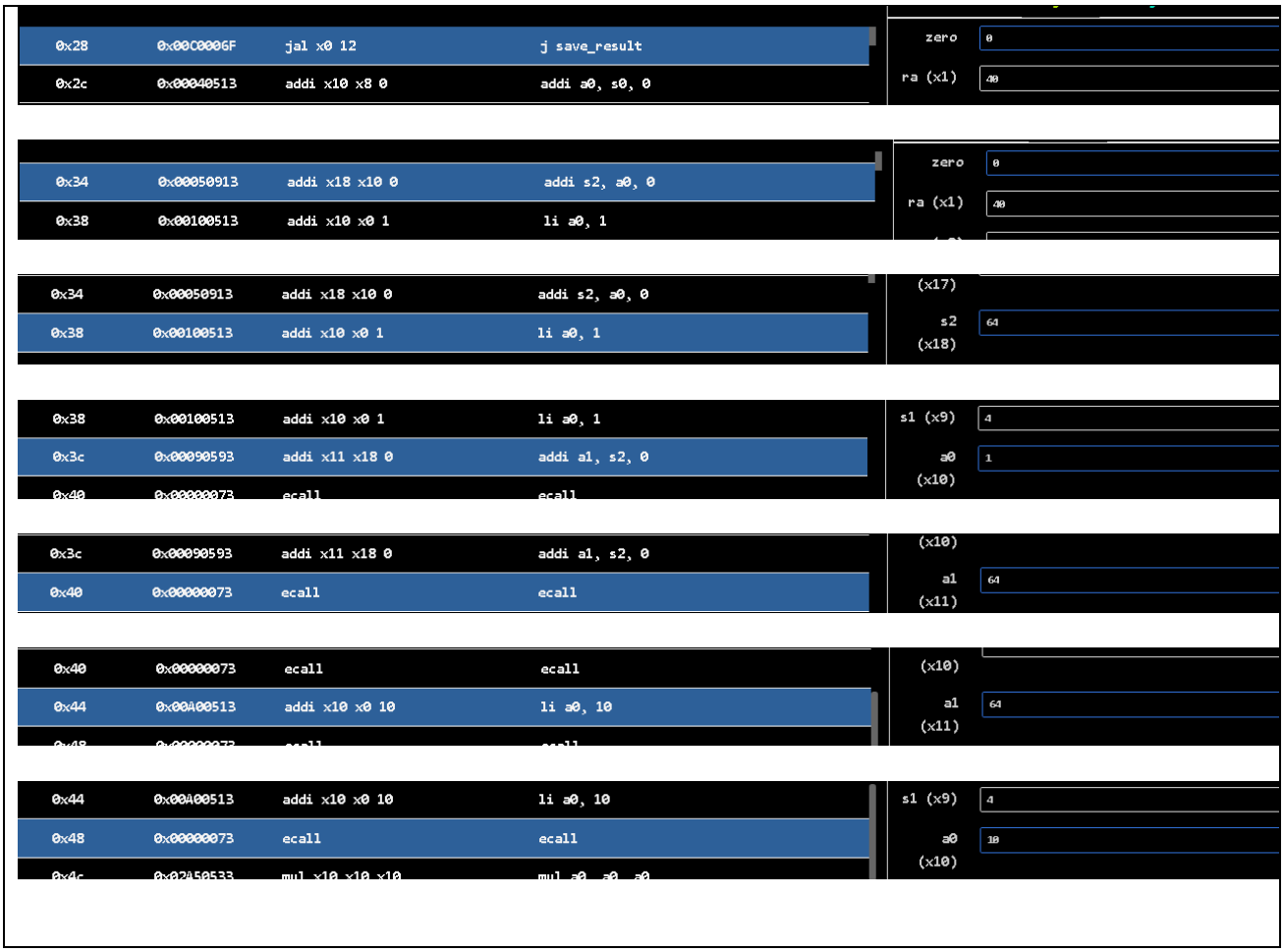
fun1:
    mul a0, a0, a0
    jalr zero, ra, 0

fun2:
    mul t0, a0, a0
    mul a0, t0, a0
    jalr zero, ra, 0
```

```
fun3:
   addi a0, a0, 1
   jalr zero, ra, 0
```

Output:





Task 8: Implement the following C code in RISC-V assembly using Venus.

Description:
In this task, we implemented the Fibonacci sequence using both iterative and recursive procedures in RISC-V assembly. The iterative version uses a loop, while the recursive version calls itself and uses the stack to preserve return addresses and intermediate results. We used n=7, with F(0)=0 and F(1)=1. Both implementations produce the final result 13.
Code:
Without Recursion:

```
.text

.globl main

main:

   li a0, 7

   jal ra, fib

   addi s0, a0, 0

   li a0, 1

   addi a1, s0, 0

   ecall

   li a0, 10

   ecall

fib:

   beq a0, zero, fib_zero

   li t0, 1

   beq a0, t0, fib_one

   li t0, 0

   li t1, 1

   li t2, 2

fib_loop:

   bgt t2, a0, fib_done

   add t3, t0, t1

   addi t0, t1, 0

   addi t1, t3, 0
```

```
    addi t2, t2, 1
    j fib_loop
fib_done:
    addi a0, t1, 0
    jalr zero, ra, 0
fib_zero:
    li a0, 0
    jalr zero, ra, 0
fib_one:
    li a0, 1
    jalr zero, ra, 0
```

With Recursion:

```
.text
.globl main
main:
    li a0, 7
    jal ra, fib
    addi s0, a0, 0
    li a0, 1
    addi a1, s0, 0
    ecall
    li a0, 10
    ecall
```

```
fib:
    beq a0, zero, fib_zero
    li t0, 1
    beq a0, t0, fib_one

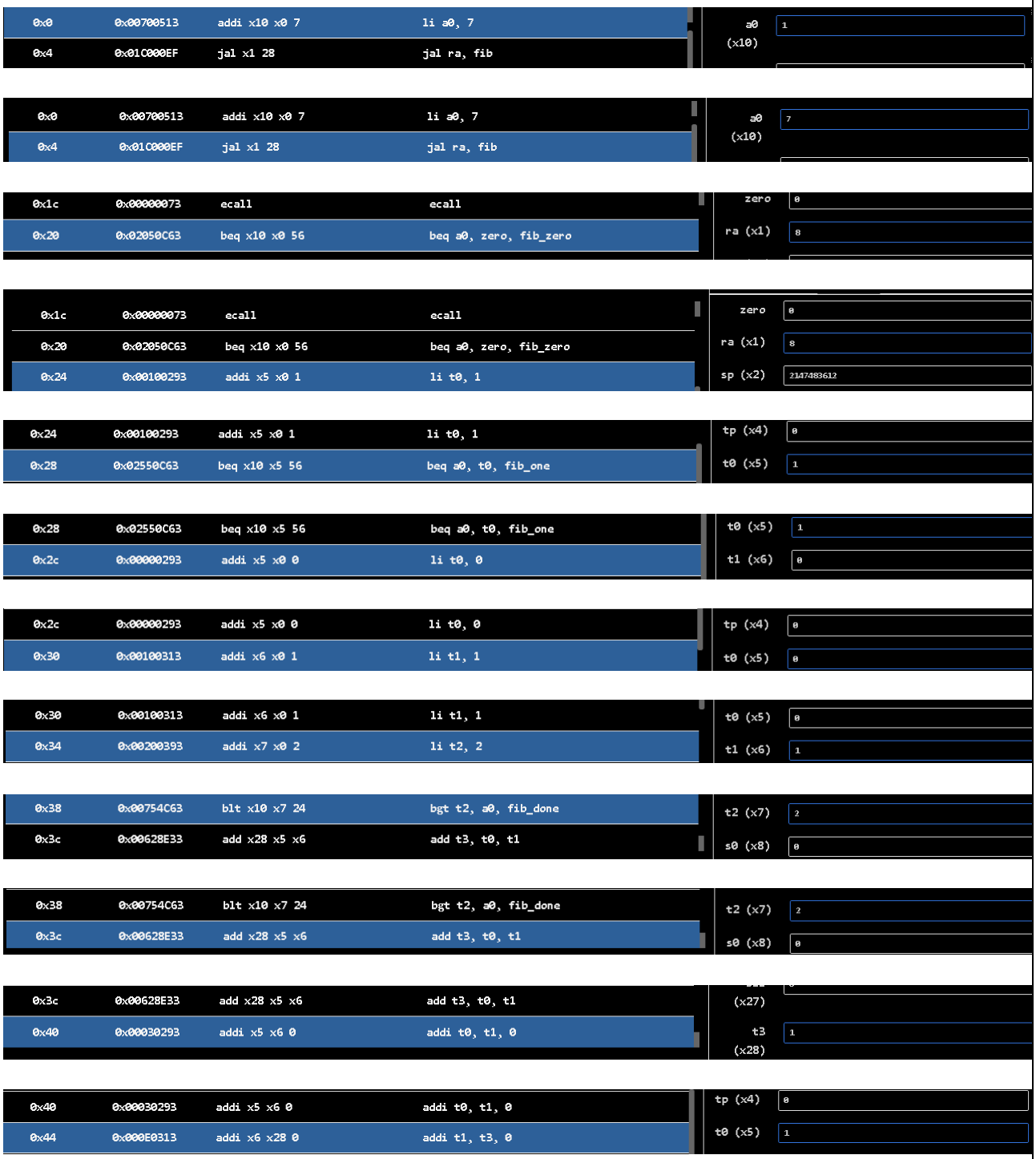

    addi sp, sp, -16
    sw ra, 12(sp)
    sw a0, 8(sp)
    addi a0, a0, -1
    jal ra, fib
    sw a0, 4(sp)
    lw a0, 8(sp)
    addi a0, a0, -2
    jal ra, fib
    lw t0, 4(sp)
    add a0, t0, a0
    lw ra, 12(sp)
    addi sp, sp, 16
    jalr zero, ra, 0
fib_zero:
    li a0, 0
    jalr zero, ra, 0
fib_one:
```

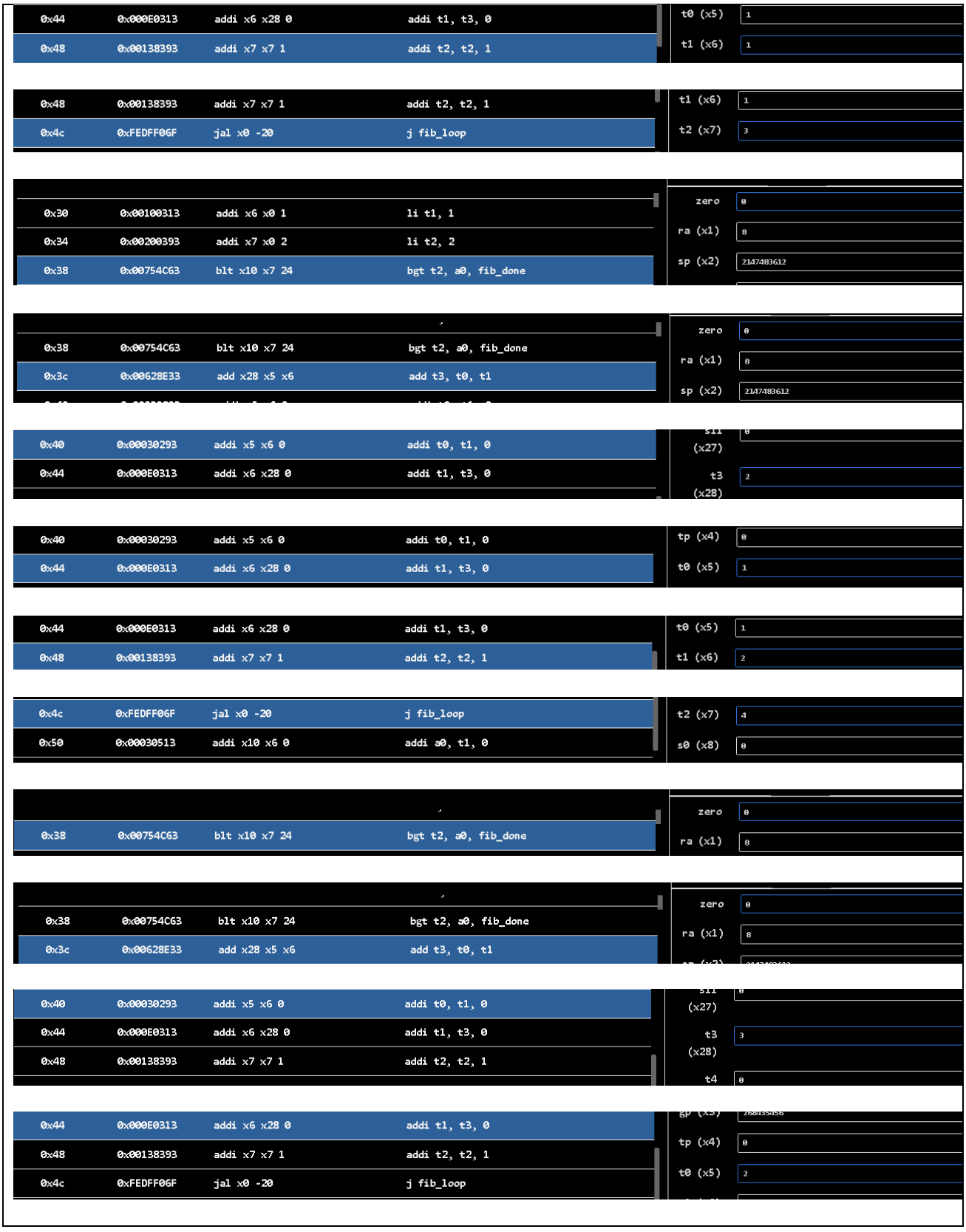
li a0, 1

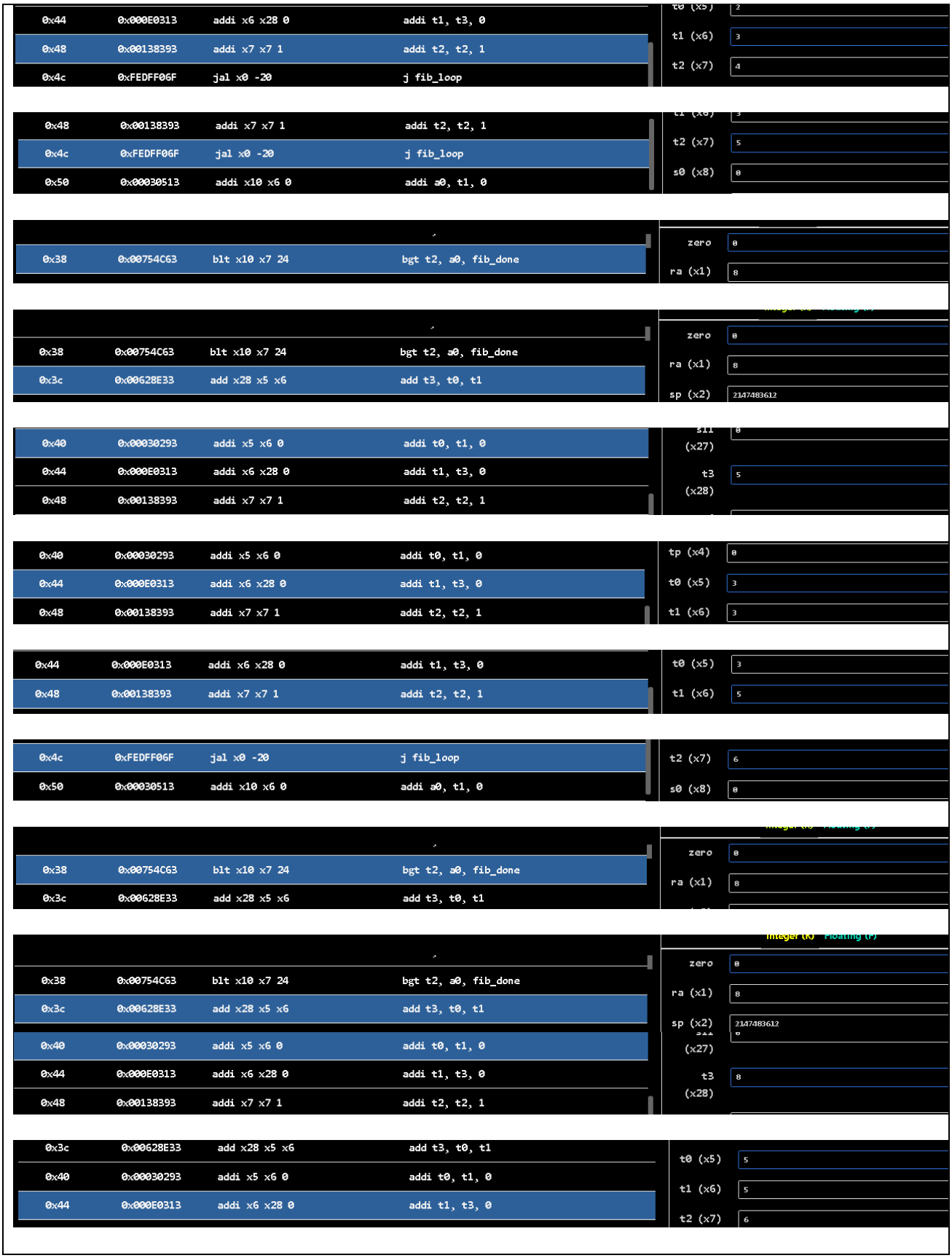
jalr zero, ra, 0

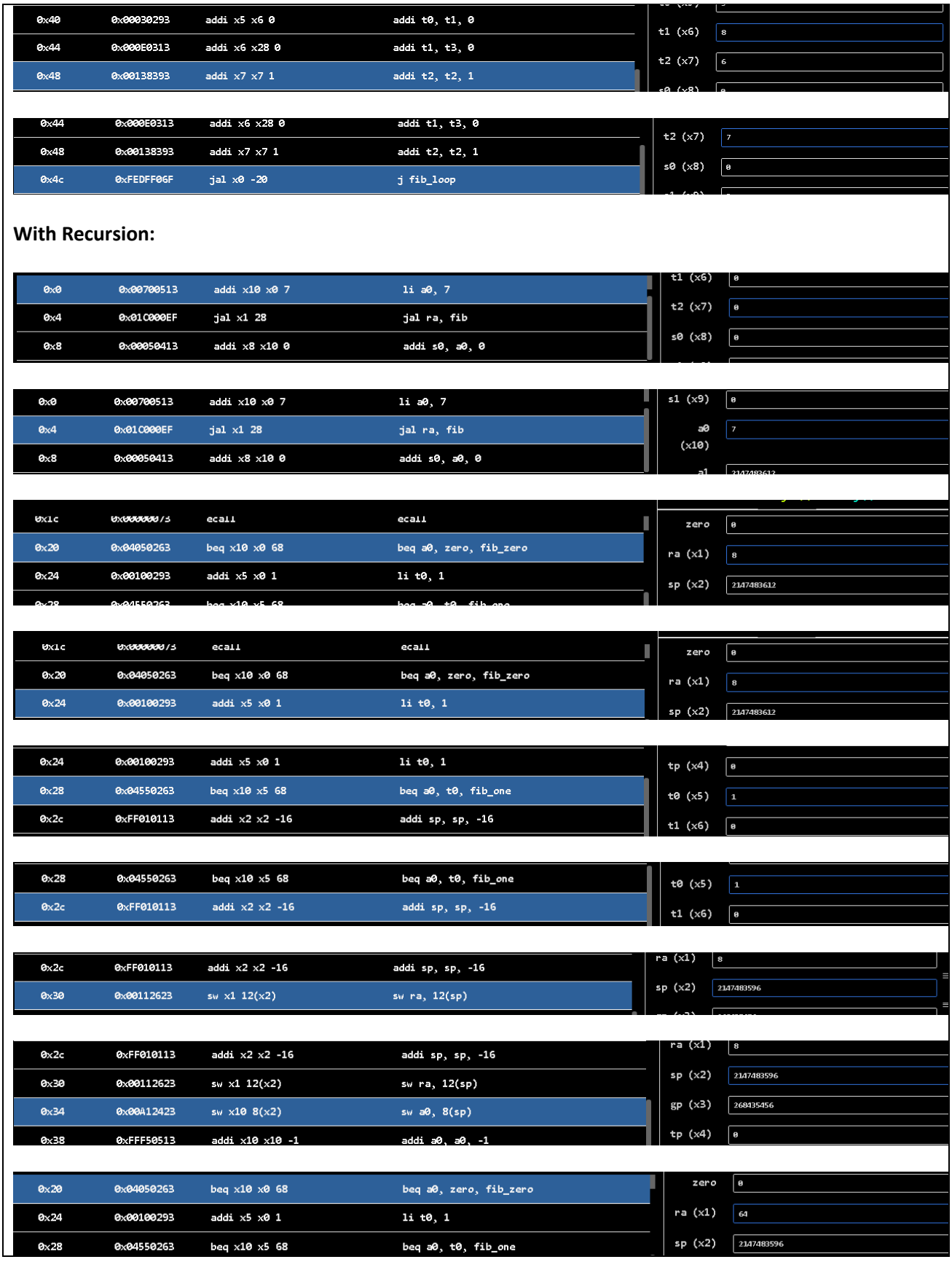
Output:

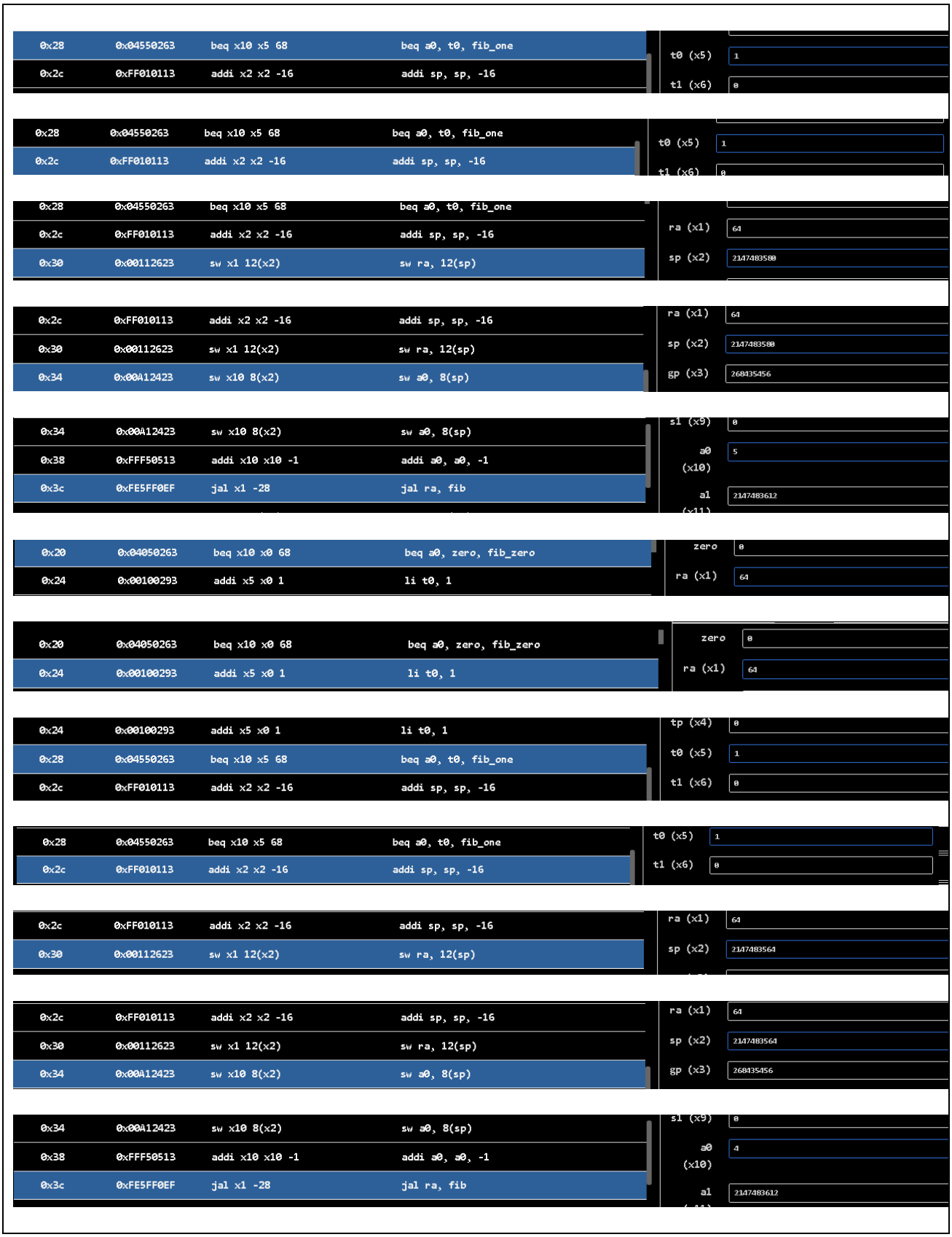
Without Recursion:

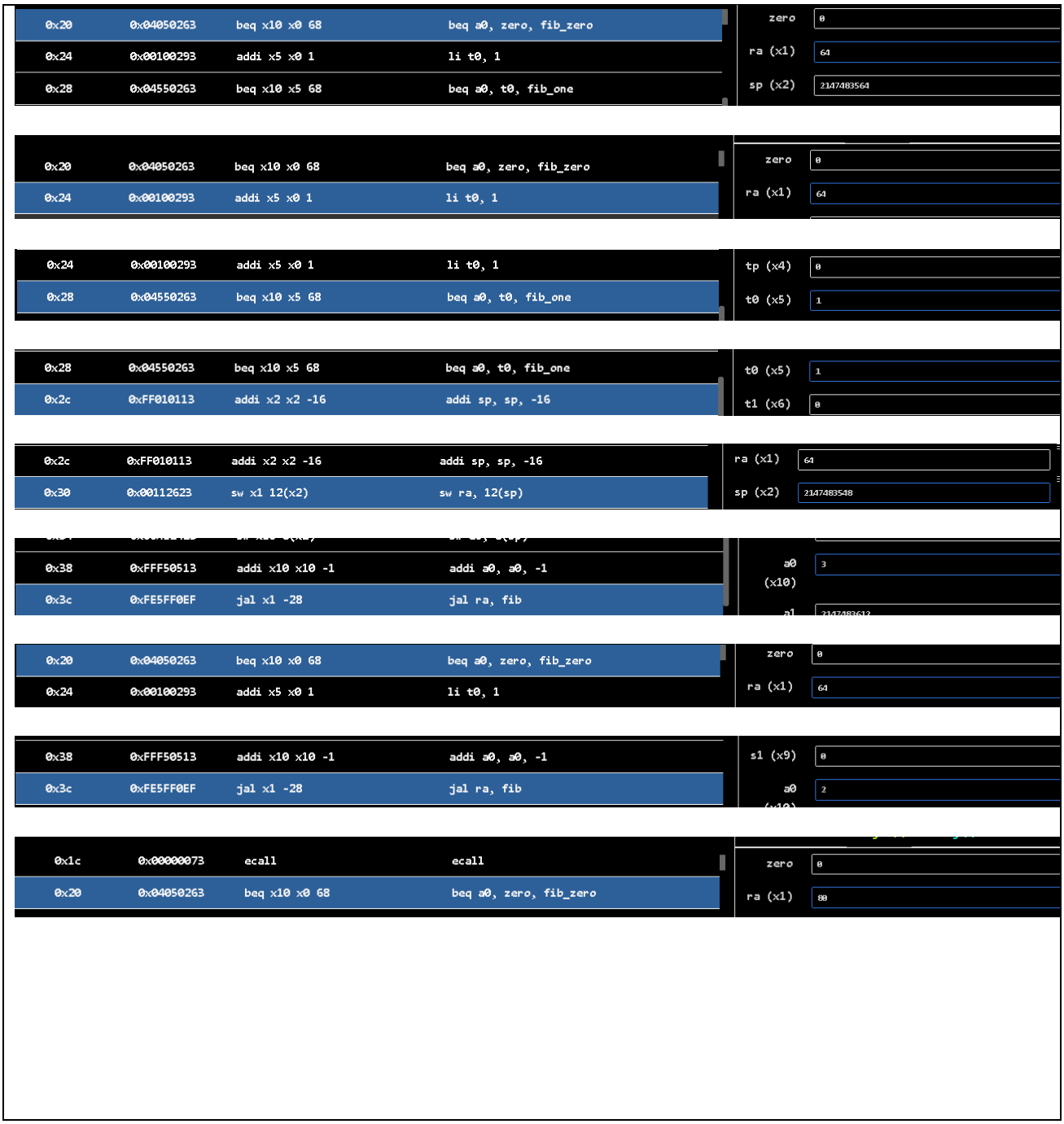



0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
t0 (x5) 1
t1 (x6) 1
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
0x4c 0xFEDFF06F jal x0 -20 j fib_loop
t1 (x6) 1
t2 (x7) 3
0x30 0x00100313 addi x6 x0 1 li t1, 1
0x34 0x00200393 addi x7 x0 2 li t2, 2
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
zero 0
ra (x1) 8
sp (x2) 2147483612
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
0x3c 0x00628E33 add x28 x5 x6 add t3, t0, t1
zero 0
ra (x1) 8
sp (x2) 2147483612
0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
t3 (x28) 2
0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
tp (x4) 0
t0 (x5) 1
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
t0 (x5) 1
t1 (x6) 2
0x4c 0xFEDFF06F jal x0 -20 j fib_loop
0x50 0x00030513 addi x10 x6 0 addi a0, t1, 0
t2 (x7) 4
s0 (x8) 0
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
zero 0
ra (x1) 8
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
0x3c 0x00628E33 add x28 x5 x6 add t3, t0, t1
zero 0
ra (x1) 8
0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
t3 (x28) 3
t4 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
0x4c 0xFEDFF06F jal x0 -20 j fib_loop
tp (x4) 0
t0 (x5) 2


0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
0x4c 0xFEDFF06F jal x0 -20 j fib_loop
t1 (x6) 3
t2 (x7) 4
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
0x4c 0xFEDFF06F jal x0 -20 j fib_loop
0x50 0x00030513 addi x10 x6 0 addi a0, t1, 0
t2 (x7) 5
s0 (x8) 0
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
zero 0
ra (x1) 8
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
0x3c 0x00628E33 add x28 x5 x6 add t3, t0, t1
zero 0
ra (x1) 8
sp (x2) 2147483612
0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
t3 (x28) 5
0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
tp (x4) 0
t0 (x5) 3
t1 (x6) 3
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
t0 (x5) 3
t1 (x6) 5
0x4c 0xFEDFF06F jal x0 -20 j fib_loop
0x50 0x00030513 addi x10 x6 0 addi a0, t1, 0
t2 (x7) 6
s0 (x8) 0
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
0x3c 0x00628E33 add x28 x5 x6 add t3, t0, t1
zero 0
ra (x1) 8
0x38 0x00754C63 blt x10 x7 24 bgt t2, a0, fib_done
0x3c 0x00628E33 add x28 x5 x6 add t3, t0, t1
0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
zero 0
ra (x1) 8
sp (x2) 2147483612
t3 (x28) 8
0x3c 0x00628E33 add x28 x5 x6 add t3, t0, t1
0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
t0 (x5) 5
t1 (x6) 5
t2 (x7) 6


0x40 0x00030293 addi x5 x6 0 addi t0, t1, 0
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
t1 (x6) 8
t2 (x7) 6
0x44 0x000E0313 addi x6 x28 0 addi t1, t3, 0
0x48 0x00138393 addi x7 x7 1 addi t2, t2, 1
0x4c 0xFEDFF06F jal x0 -20 j fib_loop
t2 (x7) 7
s0 (x8) 0
With Recursion:
0x0 0x00700513 addi x10 x0 7 li a0, 7
0x4 0x01C000EF jal x1 28 jal ra, fib
0x8 0x00050413 addi x8 x10 0 addi s0, a0, 0
t1 (x6) 0
t2 (x7) 0
s0 (x8) 0
0x0 0x00700513 addi x10 x0 7 li a0, 7
0x4 0x01C000EF jal x1 28 jal ra, fib
0x8 0x00050413 addi x8 x10 0 addi s0, a0, 0
s1 (x9) 0
a0 (x10) 7
0x1c 0x00000073 ecall ecall
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
zero 0
ra (x1) 8
sp (x2) 2147483612
0x1c 0x00000073 ecall ecall
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
zero 0
ra (x1) 8
sp (x2) 2147483612
0x24 0x00100293 addi x5 x0 1 li t0, 1
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
tp (x4) 0
t0 (x5) 1
t1 (x6) 0
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
t0 (x5) 1
t1 (x6) 0
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
0x30 0x00112623 sw x1 12(x2) sw ra, 12(sp)
ra (x1) 8
sp (x2) 2147483596
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
0x30 0x00112623 sw x1 12(x2) sw ra, 12(sp)
0x34 0x00A12423 sw x10 8(x2) sw a0, 8(sp)
0x38 0xFFF50513 addi x10 x10 -1 addi a0, a0, -1
ra (x1) 8
sp (x2) 2147483596
gp (x3) 268435456
tp (x4) 0
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
zero 0
ra (x1) 64
sp (x2) 2147483596


0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
t0 (x5) 1
t1 (x6) 0
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
t0 (x5) 1
t1 (x6) 0
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
0x30 0x00112623 sw x1 12(x2) sw ra, 12(sp)
ra (x1) 64
sp (x2) 2147483580
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
0x30 0x00112623 sw x1 12(x2) sw ra, 12(sp)
0x34 0x00A12423 sw x10 8(x2) sw a0, 8(sp)
ra (x1) 64
sp (x2) 2147483580
gp (x3) 268435456
0x34 0x00A12423 sw x10 8(x2) sw a0, 8(sp)
0x38 0xFFF50513 addi x10 x10 -1 addi a0, a0, -1
0x3c 0xFE5FF0EF jal x1 -28 jal ra, fib
s1 (x9) 0
a0 (x10) 5
a1 2147483612
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
zero 0
ra (x1) 64
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
zero 0
ra (x1) 64
0x24 0x00100293 addi x5 x0 1 li t0, 1
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
tp (x4) 0
t0 (x5) 1
t1 (x6) 0
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
t0 (x5) 1
t1 (x6) 0
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
0x30 0x00112623 sw x1 12(x2) sw ra, 12(sp)
ra (x1) 64
sp (x2) 2147483564
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
0x30 0x00112623 sw x1 12(x2) sw ra, 12(sp)
0x34 0x00A12423 sw x10 8(x2) sw a0, 8(sp)
ra (x1) 64
sp (x2) 2147483564
gp (x3) 268435456
0x34 0x00A12423 sw x10 8(x2) sw a0, 8(sp)
0x38 0xFFF50513 addi x10 x10 -1 addi a0, a0, -1
0x3c 0xFE5FF0EF jal x1 -28 jal ra, fib
s1 (x9) 0
a0 (x10) 4
a1 2147483612


0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
zero 0
ra (x1) 64
sp (x2) 2147483564
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
zero 0
ra (x1) 64
0x24 0x00100293 addi x5 x0 1 li t0, 1
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
tp (x4) 0
t0 (x5) 1
0x28 0x04550263 beq x10 x5 68 beq a0, t0, fib_one
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
t0 (x5) 1
t1 (x6) 0
0x2c 0xFF010113 addi x2 x2 -16 addi sp, sp, -16
0x30 0x00112623 sw x1 12(x2) sw ra, 12(sp)
ra (x1) 64
sp (x2) 2147483548
0x38 0xFFF50513 addi x10 x10 -1 addi a0, a0, -1
0x3c 0xFE5FF0EF jal x1 -28 jal ra, fib
a0 (x10) 3
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
0x24 0x00100293 addi x5 x0 1 li t0, 1
zero 0
ra (x1) 64
0x38 0xFFF50513 addi x10 x10 -1 addi a0, a0, -1
0x3c 0xFE5FF0EF jal x1 -28 jal ra, fib
s1 (x9) 0
a0 2
0x1c 0x00000073 ecall ecall
0x20 0x04050263 beq x10 x0 68 beq a0, zero, fib_zero
zero 0
ra (x1) 80